```
> coffee word_utils.spec.coffee
# No output
```

Agtron, peering over your shoulder, asks if he can try something. Agreeing, you hand him the keyboard. You anticipate that Agtron is going to show you how to improve your test. He changes the test to run against multiple input values:

```
do removeWordShouldRemoveOneWord = ->
  tests = [
    initial: "product special"
    replace: "special"
    expected: "product"
  ,
    initial: "product special"
    replace: "spec"
    expected: "product special"
  ]

  for test in tests
    actual = removeWord(test.initial, test.replace)
    assert.equal actual, test.expected
```

An array of two test cases

A for…in loop to execute the array of test cases

The test fails:

```
> coffee word_utils.spec.coffee
#AssertionError: "product ial" == "product special"
```

Agtron hasn't just shown you how to improve the test; he's shown you that the test was insufficient. You need a better `removeWord` function. In chapter 3 you learned to split a string of words into an array of words. With this technique, you have a new solution:

```
removeWord = (text, toRemove) ->
  words = text.split /\s/
  (word for word in words when word isnt toRemove).join ' '
```

You now have a passing test:

```
> coffee word_utils.spec.coffee
# No output
```